CMPE3209 Computer Organization — Proje Teslim Tutanağı

Kullanım talimatı

Bu tutanak projenin tek resmi teslim belgesidir. Her kilometre taşında (M1, M2 ve final; tümü pazartesi) ilgili adım doldurulur ve aynı dosya güncellenerek yeniden teslim edilir. Ayrıntılı görev tanımları proje yönergesindedir; bu tutanak yalnızca ne istendiğini ve nereye yazılacağını gösterir.

- Tüm sayılar birimiyle yazılır (byte, cycle, ns, GHz). Hex değerler 0x önekiyle yazılır.
- **Varsayım** olan her değer "(varsayım)" etiketiyle işaretlenir.
- Tablolara sığmayan çıktılar (assembly listesi, objdump, CSV izi, ekran görüntüleri) Git deposuna konur. Tutanakta dosya adıyla atıf yapılır.
- Boş bırakılan zorunlu alan, o adımın puanını sıfırlar.

Öğrenci bilgileri

Alan	Değer
Ad Soyad	Ali Paşaoğlu
Öğrenci numarası (SID)	5001230022 (32 bite sığmadığı için LSB'den MSB'ye yerleştirilen alt 32 bit: 706262726 = 0x2A18B6C6)
E-posta	alipasaoglu@ogr.iu.edu.tr
Seçilen görev (G1–G5)	G3 — Mesaj şifreleme ve bütünlük
Seçilen ISA ve sürümü (ör. RV32IM)	RISC-V RV32IM, ABI ilp32
Derleyici ve sürümü	riscv64-unknown-elf-gcc 8.3.0 (SiFive freedom-tools 2020.04.1, Windows x86-64); -march=rv32im -mabi=ilp32 -O0
Simülatör / araçlar	Ripes v2.3.1 (Single-cycle processor, RV32IM); Visual Studio (ön doğrulama)
Git deposu adresi	https://github.com/alielric/CMPE-3209-Computer-Organization

Adım 0 — Kişiye özel parametreler

Bu adım M1 ile birlikte teslim edilir. Hatalı bir parametre, ona dayanan tüm adımları geçersiz kılar; öğretim üyesi bu tabloyu M1'de kontrol eder.

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
d	SID'in son hanesi	500123002[2]	2
D2	SID'in son iki hanesi	50012300[22]	22
S	SID'in hanelerinin toplamı	5+0+0+1+2+3+0+0+2+2	15

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
N	24 + (D2 mod 25)	24 + (22 mod 25) = 24 + 22	46 eleman (184 byte)
Cache	d'ye göre: boyut / line / mapping / line sayısı / set sayısı	d = 2 satırı	256 byte / 16 byte / direct / 16 line / set yok
Write policy	d çift → write-back, d tek → write-through	d = 2 çift	write-back
k	(S mod 7) + 3	(15 mod 7) + 3 = 1 + 3	4
Interrupt türü	S çift → Timer, S tek → I/O	S = 15 tek	I/O
f	1.0 + 0.1 × d GHz (varsayım)	1.0 + 0.1 × 2 = 1.2; τ = 1 / 1.2 GHz	1.2 GHz, τ ≈ 0.833 ns (varsayım)
T1 / T2	T1 = 1 cycle; T2 = 20 + 5 × (d mod 4) cycle (varsayım)	T2 = 20 + 5 × (2 mod 4) = 20 + 10	T1 = 1 cycle, T2 = 30 cycle (varsayım)
Nc	2 + (S mod 7) (varsayım)	2 + (15 mod 7) = 2 + 1	3 (varsayım)
Görev parametresi	Seçilen göreve göre (T, P, K, yön veya R)	K = SID mod 256 = 706262726 mod 256 (0x2A18B6C6'nın en düşük byte'ı)	K = 198 = 0xC6 (0 değil, 0xA5 gerekmez)

Not: Numara (5001230022) ikilik tabanda 33 bittir ve 32 bite sığmaz. Duyurudaki kurala göre LSB'den MSB'ye doğru yerleştirilerek alt 32 biti kullanılmıştır: 5001230022 − 2^{32} = 706262726 = 0x2A18B6C6. K bu değerden hesaplanmıştır. d, D2 ve S yönergedeki örnek hesapta olduğu gibi numaranın onluk hanelerinden hesaplanmıştır.

Adres alanları (ISA'nın adres genişliği s + w bit; 64-bit ISA'da düşük 32 bit kullanılır):

Alan	Bit sayısı	Hesap
Word (w)	4	line = 16 byte = 2^4 → w = 4
Line (r) veya set (d)	4 (line, r)	direct mapping, 16 line = 2^4 → r = 4
Tag (s − r veya s − d)	24	s = 32 − w = 28; s − r = 28 − 4 = 24
Toplam	32	24 + 4 + 4 = 32 bit (RV32 adresi)

Adım 1 — Görev seçimi ve C programı (M1: 2 Kasım, Pazartesi)

1.1 Görev seçimi (yalnızca birini işaretleyin):

- ☐ G1 Biyomedikal sinyal alarmı
- ☐ G2 Sınav notu dağılımı
- ☒ **G3 Mesaj şifreleme ve bütünlük**
- ☐ G4 Sıralama ve medyan
- ☐ G5 Run-length sıkıştırma

1.2 Çözüm tasarımı (3–5 cümle: hangi algoritma, hangi fonksiyonlar, hangi veri yerleşimi ve neden):

> *Girdi dizisi, yönergedeki LCG üretecini çalıştıran lcg_next() fonksiyonuyla SID'den (x_0 = 706262726) üretilir ve 46 elemanlı global msg[] dizisine 'A'–'Z' aralığında yazılır. Dizi K = 198 (0xC6) anahtarıyla yerinde XOR'lanarak şifrelenir; şifreli dizinin checksum'ı, 32-bit sola döndürmeyi yapan rotl1() fonksiyonuyla $c \leftarrow rotl(c, 1) + v_i$ biçiminde hesaplanıp checksum değişkenine yazılır. Ardından dizi aynı anahtarla tekrar XOR'lanarak çözülür ($(x \oplus K) \oplus K = x$). Karşılaştırma için orijinalin kopyası tutulmaz; üreteç SID'den yeniden çalıştırılır ve sonuç decrypt_ok değişkenine yazılır. Böylece bellekte tek bir dizi bulunur ve cache izi yalnızca msg[] ile stack erişimlerinden oluşur.*

1.3 C kaynak kodu. Satır numaralı kod aşağıdaki bloka yapıştırılır (25–60 satır):

```
1  /* Ali Pasaoglu, 5001230022, G3 Mesaj sifreleme ve butunluk */
2  #define N 46                    /* N = 24 + (22 mod 25) */
3
4  int msg[N];                     /* mesaj dizisi ('A'-'Z') */
5  unsigned int checksum;          /* sonuc 1 */
6  int decrypt_ok;                 /* sonuc 2 */
7
8  unsigned int lcg_next(unsigned int x) {
9      return 1664525u * x + 1013904223u;
10 }
11
12 unsigned int rotl1(unsigned int c) {
13     return (c << 1) | (c >> 31);
14 }
15
16 int main(void) {
17     const int SID = 706262726;      /* 5001230022 alt 32 bit: 0x2A18B6C6 */
18     int key = SID % 256;            /* K = 198 = 0xC6 */
19     if (key == 0)
20         key = 0xA5;
21     unsigned int x = SID;
22     int i;
23     for (i = 0; i < N; i++) {       /* 1. gecis: girdi uret */
24         x = lcg_next(x);
25         msg[i] = (int)((x >> 16) % 26) + 65;
26     }
27     for (i = 0; i < N; i++)         /* 2. gecis: sifrele */
28         msg[i] = msg[i] ^ key;
29     unsigned int c = 0;
30     for (i = 0; i < N; i++)         /* 3. gecis: checksum */
31         c = rotl1(c) + (unsigned int)msg[i];
32     checksum = c;
33     for (i = 0; i < N; i++)         /* 4. gecis: sifreyi coz */
34         msg[i] = msg[i] ^ key;
35     x = SID;                        /* ureteci bastan calistir */
36     decrypt_ok = 1;
37     for (i = 0; i < N; i++) {       /* 5. gecis: orijinalle karsilastir */
38         x = lcg_next(x);
```

```
39          if (msg[i] != (int)((x >> 16) % 26) + 65)
40              decrypt_ok = 0;
41      }
42      return 0;
43  }
```

1.4 Zorunlu yapı eşlemesi. Her madde için satır numarası ve tek cümlelik açıklama:

#	Zorunlu yapı	Satır no	Açıklama
Y1	N elemanlı global int dizisi, en az iki geçiş	4; 23–41	46 elemanlı global int msg[] dizisi üzerinde beş ayrı for döngüsüyle geçiş yapılır.
Y2	Koşullu dallanma	19, 39	Anahtarın 0 olup olmadığı ve çözülen elemanın orijinalle aynı olup olmadığı if ile denetlenir.
Y3	Parametreli, değer döndüren fonksiyon çağrısı	8–10, 12–14; çağrı 24, 31, 38	lcg_next(x) ve rotl1(c) unsigned parametre alıp değer döndürür ve jal ile çağrılır.
Y4	Toplama/çıkarma, çarpma veya shift, bit işlemi	9, 13, 25, 28	LCG'de çarpma ve toplama, rotl1'de shift ve OR, harf üretiminde shift ve mod, şifrelemede XOR yapılır.
Y5	Dizi elemanına store	25, 28, 34	msg[i]'ye üretilen, şifrelenen ve çözülen değer yazılır (sw).
Y6	const int SID ve hesapta kullanımı	17; kullanım 18, 21, 35	const int SID = 706262726 hem K = SID mod 256 anahtarını hem de üretecin başlangıç değerini belirler.
Y7	Sonucun tutulduğu global değişken(ler)	5–6; atama 32, 36, 40	Sonuçlar global checksum ve decrypt_ok değişkenlerinde tutulur.

1.5 Program çıktıları. Simülatörde çalıştırıldıktan sonra bellekten okunan değerler:

Global değişken	Adres (hex)	Değer	Ekran görüntüsü dosyası
checksum	0x00011D0C	0x255D2CDD (626 863 325)	mem_checksum.png
decrypt_ok	0x00011C50	0x00000001 (1, çözme başarılı)	mem_decrypt_ok.png
msg[0] (çözülmüş)	0x00011C54	0x00000054 ('T')	mem_decrypt_ok.png
msg[3] (kernel elemanı)	0x00011C60	0x0000004C ('L')	mem_decrypt_ok.png

Adresler gp = 0x00012000 (_start: auipc x3 0x2 + addi x3 x3 −144) üzerinden bulunmuştur: decrypt_ok = gp − 944, msg = gp − 940, checksum = gp − 756. Çözülen mesaj: TVBLGEPYHUHVASUMHGXGFNZYUJBCLDJKQYYNYUPMKNQCNI.

1.6 İzlenecek kernel (12–20 dinamik instruction; en az bir load, store, ALU, koşullu branch ve call/return içermeli):

Alan	Değer
C satır aralığı	9 (lcg_next dönüşü) ve 23–25 (1. geçiş: msg[i] yazma, i++ ve döngü koşulu)
Başlangıç ve bitiş adresi (hex)	0x0001017C (jalr x0 x1 0 = ret) → 0x00010210 … 0x00010250 (bge x15 x14 −72)
Dinamik instruction sayısı	18 (1 + 17): load ×4, store ×3, ALU ×8, remu ×1, koşullu branch ×1, return ×1
Hangi döngü iterasyonu (ör. i = 3)	i = 3 (msg[3] = 0x00011C60, değer 0x4C = 'L')

Öğretim üyesi (M1): ☐ Onaylandı ☐ Düzeltme istendi ☐ Reddedildi — Tarih: _____ Not: _____

Adım 2 — Assembly ve machine code (M2: 9 Kasım, Pazartesi)

2a. C → assembly eşlemesi (-O0; listenin tamamı depoda asm_O0.txt, C satırlarıyla açıklamalı hali asm_O0_eslesme.txt). Adresler 0x0001xxxx biçimindedir; tablo lcg_next, rotl1 ve main'deki 162 instruction'ın tamamını kapsar.

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
8–10	lcg_next(x) gövdesi: return 1664525u*x + 1013904223u	4 instr.: prolog: stack çerçevesi, x'i stack'e yaz 8 instr.: lw x; lui+addi 1664525; mul; lui+addi 1013904223; add; a0 = sonuç 3 instr.: epilog + ret	0x10144–0x10150 0x10154–0x10170 0x10174–0x1017C
12–14	rotl1(c) gövdesi: (c << 1) | (c >> 31)	4 instr.: prolog 5 instr.: lw c; slli 1; srli 31; or; a0 = sonuç 3 instr.: epilog + ret	0x10180–0x1018C 0x10190–0x101A0 0x101A4–0x101AC
16	int main(void) {	4 instr.: prolog: sp −48, ra ve s0 kaydı, s0 = çerçeve	0x101B0–0x101BC
17	const int SID = 706262726;	3 instr.: lui 0x2a18b + addi 1734 → sw −36(s0)	0x101C0–0x101C8
18	int key = SID % 256;	7 instr.: işaretli mod: srai/srli/add/andi/sub → sw −20(s0)	0x101CC–0x101E4
19	if (key == 0)	2 instr.: lw key; bne key, x0, +12	0x101E8–0x101EC
20	key = 0xA5;	2 instr.: addi 165; sw −20(s0)	0x101F0–0x101F4
21	unsigned int x = SID;	2 instr.: lw −36; sw −24(s0)	0x101F8–0x101FC
23	for (i = 0; i < N; i++) — 1. geçiş	2 instr.: i = 0; jal → koşul 3 instr.: i++	0x10200–0x10204 0x1023C–0x10244

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
		3 instr.: i ≤ 45 ise bge ile gövdeye dön	0x10248–0x10250
24	x = lcg_next(x);	3 instr.: a0 = x; jal lcg_next; x = a0	0x10208–0x10210
25	msg[i] = (int)((x >> 16) % 26) + 65;	10 instr.: srli 16; remu 26; addi 65; &msg[i] = gp−940 + 4i; sw	0x10214–0x10238
27	for (…) — 2. geçiş (şifrele)	2 instr.: i = 0; jal → koşul 3 instr.: i++ 3 instr.: koşul, bge	0x10254–0x10258 0x1028C–0x10294 0x10298–0x102A0
28	msg[i] = msg[i] ^ key;	12 instr.: lw msg[i]; lw key; xor; sw msg[i]	0x1025C–0x10288
29	unsigned int c = 0;	1 instr.: sw x0, −32(s0)	0x102A4
30	for (…) — 3. geçiş (checksum)	2 instr.: i = 0; jal → koşul 3 instr.: i++ 3 instr.: koşul, bge	0x102A8–0x102AC 0x102D8–0x102E0 0x102E4–0x102EC
31	c = rotl1(c) + (unsigned int)msg[i];	10 instr.: a0 = c; jal rotl1; lw msg[i]; add; sw c	0x102B0–0x102D4
32	checksum = c;	2 instr.: lw c; sw −756(gp)	0x102F0–0x102F4
33	for (…) — 4. geçiş (çöz)	2 instr.: i = 0; jal → koşul 3 instr.: i++ 3 instr.: koşul, bge	0x102F8–0x102FC 0x10330–0x10338 0x1033C–0x10344
34	msg[i] = msg[i] ^ key;	12 instr.: lw; lw; xor; sw	0x10300–0x1032C
35	x = SID;	2 instr.: lw −36; sw −24	0x10348–0x1034C
36	decrypt_ok = 1;	2 instr.: addi 1; sw −944(gp)	0x10350–0x10354
37	for (…) — 5. geçiş (karşılaştır)	2 instr.: i = 0; jal → koşul 3 instr.: i++ 3 instr.: koşul, bge	0x10358–0x1035C 0x1039C–0x103A4 0x103A8–0x103B0
38	x = lcg_next(x);	3 instr.: a0 = x; jal lcg_next; x = a0	0x10360–0x10368
39	if (msg[i] != …)	11 instr.: lw msg[i]; srli; remu; addi 65; beq eşitse +8 atla	0x1036C–0x10394
40	decrypt_ok = 0;	1 instr.: sw x0, −944(gp)	0x10398
42	return 0;	2 instr.: a0 = 0	0x103B4–0x103B8
43	}	4 instr.: epilog: ra/s0 geri yükle, sp +48, ret	0x103BC–0x103C8

Bölüm	Başlangıç adresi (hex)	Boyut (byte)	İçerik
.text	0x00010074	1912 (0x10074–0x107EB)	Tüm makine kodu: başlangıç kodu (_start, memset, exit vb.) ve programın fonksiyonları: lcg_next 0x10144, rotl1 0x10180, main 0x101B0–0x103CB (toplam 162 instr. = 648 B).
.data / .bss	.data 0x00011800 .sdata 0x00011C28 .bss 0x00011C34	1064 12 220 (0x11C34–0x11D0F)	.data/.sdata: C kütüphanesinin başlangıç verileri. .bss: sıfırla başlatılan global değişkenler; _start'taki memset(gp−972, 0, 220) ile temizlenir: decrypt_ok 0x11C50, msg[46] 0x11C54–0x11D0B, checksum 0x11D0C. gp = 0x00012000.

Von Neumann ilkesi bu adreslerde nasıl görünüyor? (1–2 cümle)

> *Program kodu (.text, 0x00010074–0x000107EB) ile veri (.data 0x00011800 ve msg[] 0x00011C54) aynı 32-bit adres uzayında, yalnızca birkaç KB arayla aynı bellekte durur. İşlemci instruction'ı PC'nin gösterdiği adresten, veriyi lw/sw ile hesaplanan adresten aynı bellek arayüzü üzerinden okur. Örneğin Ripes Memory görünümünde 0x000101C0 adresinde instruction word'ü 0x2A18B7B7 (lui), 0x00011C54 adresinde veri 0x00000054 ('T') görünür.*

2b. -O0 ve -O1 karşılaştırması (kernel için):

Ölçü	-O0	-O1	Farkın nedeni
Kernel'deki instruction sayısı	18 (seçilen pencere) 34 (bir tam iterasyon, lcg_next dahil)	8 (bir tam iterasyon, 0x10198–0x101B4)	-O1'de lcg_next main'in içine alındı (inline): call/ret, prolog ve epilog yok. i sayacı yerine doğrudan adres artırılıyor (x11 += 4; bne x11, x13). Sabitler (1664525, 1013904223, 26) döngüden önce bir kez register'a yükleniyor (0x10184–0x10194).
Load sayısı	4 (pencere) 7 (iterasyon)	0	-O0 x, i ve key değişkenlerini stack'te tutar ve her kullanımda lw ile okur. -O1 hepsini register'da tutar (register allocation): x → x14, &msg[i] → x11.
Store sayısı	3 (pencere) 5 (iterasyon)	1	-O0 her atamadan sonra x ve i'yi stack'e yazar, lcg_next de çerçevesine x8 ve x10'u kaydeder. -O1'de yalnızca msg[i] = … ataması (sw x15, 0(x11)) kalır.
Kullanılan register sayısı	7: x1, x3, x8, x10, x13, x14, x15 (pencere) 8: + x2 (iterasyon)	7: x10, x11, x13, x14, x15, x16, x17	Sayılar yakın ama register'ların rolü farklı. -O0'da x2 ve x8 stack adreslemesi için kullanılır, x14 ve x15 geçici olarak dönüşümlü kullanılır. -O1'de her değer kendi register'ında kalır: x16 = 1664525, x10 = 1013904223, x17 = 26, x13 = dizinin sonu.

Not: Bir iterasyondaki bellek referansı (load + store) -O0'da 12, -O1'de 1'dir; fark tamamen register allocation'dan gelir. -O1'de de SID, lui x14, 0x2A18B + addi x14, x14, 1734 ile yüklenir (0x1017C). K = 198 ise derleyici tarafından derleme

anında hesaplanıp xori x14, x14, 198 komutunun immediate'ine yazılmıştır (sabit katlama). Tüm program için dinamik Ic ve m değerleri Aşama 5a'da simülatörden okunacaktır. -O1 listesi: asm_O1.txt.

2c. Elle machine code (12 instruction; R, I, S, B, U ve J formatlarının hepsi kullanıldı). Kaynak: RISC-V Unprivileged Specification, RV32I/M bölümleri. Her sonuç, Ripes'in gösterdiği machine code ile karşılaştırıldı (mc_dogrulama.png: # 1–6, 8, 9; mc_dogrulama2.png: # 7, 12; mc_dogrulama3.png: # 10, 11; asm_O0.txt).

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?

1	`0x000101C0`	lui x15, 0x2a18b
		(SID üst 20 bit)
			U	`imm[31:12] [31–12] = 00101010000110001011`
				`rd [11–7] = 01111`
				`opcode [6–0] = 0110111`
					`0x2A18B7B7`	`b7 b7 18 2a`
						`(little-endian)`
							Evet
							Ripes: 2a18b7b7

2	`0x000101C4`	addi x15, x15, 1734
		(SID alt 12 bit)
			I	`imm[11:0] [31–20] = 011011000110`
				`rs1 [19–15] = 01111`
				`funct3 [14–12] = 000`
				`rd [11–7] = 01111`
				`opcode [6–0] = 0010011`
					`0x6C678793`	`93 87 67 6c`
						`(little-endian)`
							Evet
							Ripes: 6c678793

3	`0x000101C8`	sw x15, -36(x8)
		(SID stack'e yazılır)
			S	`imm[11:5] [31–25] = 1111110`
				`rs2 [24–20] = 01111`
				`rs1 [19–15] = 01000`
				`funct3 [14–12] = 010`
				`imm[4:0] [11–7] = 11100`
				`opcode [6–0] = 0100011`
					`0xFCF42E23`	`23 2e f4 fc`
						`(little-endian)`
							Evet
							Ripes: fcf42e23

4	`0x000101CC`	lw x14, -36(x8)
		(load)
			I	`imm[11:0] [31–20] = 111111011100`
				`rs1 [19–15] = 01000`
				`funct3 [14–12] = 010`
				`rd [11–7] = 01110`
				`opcode [6–0] = 0000011`
					`0xFDC42703`	`03 27 c4 fd`
						`(little-endian)`
							Evet
							Ripes: fdc42703

5	`0x000101EC`	bne x15, x0, 12
		(if (key == 0))
			B	`imm[12] [31] = 0`
				`imm[10:5] [30–25] = 000000`
				`rs2 [24–20] = 00000`
				`rs1 [19–15] = 01111`
				`funct3 [14–12] = 001`
				`imm[4:1] [11–8] = 0110`
				`imm[11] [7] = 0`
					`0x00079663`	`63 96 07 00`
						`(little-endian)`
							Evet
							Ripes: 00079663

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?
				opcode [6–0] = 1100011			
6	0x0001020C	jal x1, -200 <lcg_next> (fonksiyon çağrısı)	J	imm[20] [31] = 1 imm[10:1] [30–21] = 1110011100 imm[11] [20] = 1 imm[19:12] [19–12] = 11111111 rd [11–7] = 00001 opcode [6–0] = 1101111	0xF39FF0EF	ef f0 9f f3 (little-endian)	Evet Ripes: f39ff0ef
7	0x00010160	mul x14, x14, x15 (LCG çarpması (M uzantısı))	R	funct7 [31–25] = 0000001 rs2 [24–20] = 01111 rs1 [19–15] = 01110 funct3 [14–12] = 000 rd [11–7] = 01110 opcode [6–0] = 0110011	0x02F70733	33 07 f7 02 (little-endian)	Evet Ripes: 02f70733
8	0x00010218	srli x14, x15, 16 (x >> 16 (kernel))	I	funct6 (imm[11:6]) [31–26] = 000000 shamt [25–20] = 010000 rs1 [19–15] = 01111 funct3 [14–12] = 101 rd [11–7] = 01110 opcode [6–0] = 0010011	0x0107D713	13 d7 07 01 (little-endian)	Evet Ripes: 0107d713
9	0x00010238	sw x14, 0(x15) (msg[i] = ... (kernel, Y5))	S	imm[11:5] [31–25] = 0000000 rs2 [24–20] = 01110 rs1 [19–15] = 01111 funct3 [14–12] = 010 imm[4:0] [11–7] = 00000 opcode [6–0] = 0100011	0x00E7A023	23 a0 e7 00 (little-endian)	Evet Ripes: 00e7a023
10	0x00010250	bge x15, x14, -72 (döngü koşulu (kernel))	B	imm[12] [31] = 1 imm[10:5] [30–25] = 111101 rs2 [24–20] = 01110 rs1 [19–15] = 01111 funct3 [14–12] = 101 imm[4:1] [11–8] = 1100	0xFAE7DCE3	e3 dc e7 fa (little-endian)	Evet Ripes: fae7dce3

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?
				`imm[11] [7] = 1` `opcode [6–0] = 1100011`			

11	`0x00010274`	xor x14, x14, x15
		(msg[i] ^ key)
			R	`funct7 [31–25] = 0000000`
				`rs2 [24–20] = 01111`
				`rs1 [19–15] = 01110`
				`funct3 [14–12] = 100`
				`rd [11–7] = 01110`
				`opcode [6–0] = 0110011`
					`0x00F74733`	`33 47 f7 00`
						`(little-endian)`
							Evet
							Ripes: 00f74733

12	`0x0001017C`	jalr x0, 0(x1)
		(ret (kernel başı))
			I	`imm[11:0] [31–20] = 000000000000`
				`rs1 [19–15] = 00001`
				`funct3 [14–12] = 000`
				`rd [11–7] = 00000`
				`opcode [6–0] = 1100111`
					`0x00008067`	`67 80 00 00`
						`(little-endian)`
							Evet
							Ripes: 00008067

Zorunlu instruction'ların tablodaki sıra numarası:

- ☒ SID'i yükleyen instruction(lar): # 1 (lui) ve # 2 (addi)
- ☒ Koşullu branch: # 5 (bne, ileri) ve # 10 (bge, geri)
- ☒ Load: # 4 (lw)
- ☒ Store: # 3 ve # 9 (sw)
- ☒ Fonksiyon çağrısı: # 6 (jal lcg_next); dönüş # 12 (jalr = ret)

SID'in yüklenmesi. Sabitin neden tek instruction'a sığmadığı, nasıl bölündüğü ve gerekiyorsa +1 düzeltmenin sign extension ile açıklaması:

> *SID = 706262726 = 0x2A18B6C6 sayısının 30 anlamlı biti var. I-type addi'nin immediate'i ise yalnızca 12 bit (−2048 … +2047) olduğundan sabit tek instruction'a sığmaz ve ikiye bölünür: (1) lui x15, 0x2A18B üst 20 biti yükler, alt 12 biti sıfırlar → x15 = 0x2A18B000. (2) addi x15, x15, 0x6C6 (= 1734) alt 12 biti ekler → x15 = 0x2A18B6C6. addi'nin 12-bit immediate'i toplanmadan önce 32 bite işaretle genişletilir (sign extension). Alt 12 bitin en üst biti (bit 11) 1 ise immediate negatif sayılır (değer − 4096); bu farkı kapatmak için lui'ye yazılan üst kısım 1 artırılır: üst = (SID + 0x800) >> 12. Bizim alt 12 bitimiz 0x6C6 = 0110 1100 0110_2; bit 11 = 0 olduğu için immediate pozitiftir ve +1 düzeltmesi gerekmez: (0x2A18B6C6 + 0x800) >> 12 = 0x2A18B. Karşı örnek: alt 12 bit 0x9C6 olsaydı imm = 0x9C6 − 0x1000 = −1594 olur, lui'ye 0x2A18C yazılırdı (0x2A18C000 − 1594 = 0x2A18B9C6).*

Branch offset hesabı. Branch'in adresi, hedef adres ve offset'in adım adım hesabı:

> *İleri branch (# 5): bne x15, x0, 12, adres 0x000101EC (C 19: key 0 değilse 20. satırı atla). Hedef 0x000101F8'deki lw x15, −36(x8) (C 21). Offset = hedef − PC = 0x101F8 − 0x101EC = 0xC = +12 byte. B-type offset 13 bit işaretli bir sayıdır ve en düşük biti hep 0 olduğundan saklanmaz: +12 = 0000000001100_2 → imm[12] = 0, imm[11] = 0, imm[10:5] = 000000, imm[4:1] = 0110 → 0x00079663. Geri branch (# 10): bge x15, x14, −72, adres 0x00010250 (1. geçiş döngü koşulu, 45 ≥ i). Hedef 0x00010208 (döngü gövdesinin başı, C 24). Offset = 0x10208 − 0x10250 = −0x48*

> *= −72. 13 bit ikiye tümleyen: +72 = 0 0000 0100 1000_2 → tersi + 1 = 1 1111 1011 1000_2 (0x1FB8) → imm[12] = 1, imm[11] = 1, imm[10:5] = 111101, imm[4:1] = 1100 → 0xFAE7DCE3. Branch alınırsa yeni PC = PC + offset olur; alınmazsa PC + 4 olur.*

Adım 3 — CPU, interrupt ve bus (final paketiyle: 16 Kasım, Pazartesi)

Model varsayımları (okudum ve uyguladım):

- ☐ Pipeline yok, her an tek instruction yürütülür.
- ☐ PC, IR, MAR ve MBR Fig. 3.2'deki gibi kavramsal olarak vardır.
- ☐ Her bellek erişimi bir bus işlemidir; cache bu adımda yok sayılır.
- ☐ Data bus genişliği = ____ bit.

Modelin gerçek donanımdan ayrıldığı yerler (3–5 cümle):

>

3a. Instruction cycle izleme tablosu. Her dinamik instruction bir satırdır. Bir instruction birden çok bellek erişimi yapıyorsa MAR, MBR ve bus hücrelerine sırayla yazılır. Kategori: PM = processor–memory, PIO = processor–I/O, DP = data processing, C = control.

#	PC (önce → sonra)	Assembly	IR (hex)	Fig. 3.6 durumları	MAR	MBR	ALU işlemi	Değişen register	Bus (adres / data / control)	Kategori
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										
11										
12										
13										
14										
15										

#	PC (önce → sonra)	Assembly	IR (hex)	Fig. 3.6 durumları	MAR	MBR	ALU işlemi	Değişen register	Bus (adres / data / control)	Kategori
16										
17										
18										
19										
20										

Kernel toplamları	Sayı
Instruction fetch	
Operand fetch (of)	
Operand store (os)	
PC'nin sıra dışı değiştiği instruction	

3b. Fig. 3.4 makinesine aktarım. Kernel'in tek bir dizi elemanını güncelleyen parçası, opcode 1, 2, 3, 5 ve 7 ile yeniden yazılır ve Fig. 3.5 formatında izlenir:

Adım	Fetch / execute	PC	IR (hex)	AC	Değişen bellek hücresi
1					
2					
3					
4					
5					
6					
7					
8					

Ölçü	Seçilen ISA	Fig. 3.4 makinesi
Aynı iş için instruction cycle sayısı		

Tek accumulator'ın getirdiği sınırlama (PDP-11 `ADD B,A` ile karşılaştırarak):

IAS register'ları ve seçilen ISA:

IAS register'ı	IAS'taki görevi	Seçilen ISA'daki karşılığı	Programcıya görünür mü?
MBR			
MAR			
IR			
IBR			
PC			
AC			
MQ			

3c. Interrupt senaryosu (k = ____, tür = ____):

Soru	Cevap
Interrupt hangi instruction'ın execute'u sırasında geliyor? (adres, assembly)	
Ne zaman tanınır ve bu instruction neden yarıda kesilmez? (Fig. 3.9)	
Kaydedilen PC değeri (hex)	
Kaydedilen diğer durum bilgisi	
Seçilen ISA'daki gerçek mekanizma (register adları, vektör)	
Kaynak (ISA referansı, bölüm, sayfa)	
Handler dönüşünde ilk çalışan instruction'ın adresi	

Fonksiyon çağrısı (Y3) ve interrupt karşılaştırması:

Ölçüt	Fonksiyon çağrısı	Interrupt
Akışı kim başlatır?		
Dönüş adresini kim kaydeder, nereye?		
User program bundan haberdar mı?		

İkinci (daha yüksek öncelikli) interrupt. Aynı olay dizisini iki yaklaşımla zaman çizelgesi olarak doldurun (Fig. 3.13 ve 3.14):

Zaman	Sequential (disable)	Nested (priority)
t_0: 1. interrupt gelir		
t_1: 2. interrupt gelir		
t_2		

Zaman	Sequential (disable)	Nested (priority)
t_3		
t_4: user program'a dönüş		

3d. Bus ve interconnection.

Bus işlemi türü (kernel, cache yok)	Sayı	Aktif control line'lar
Instruction fetch (read)		
Data read		
Data write		
Toplam		

Hesap	Değer	Adımlar
Address bus genişliği ve maksimum adreslenebilir bellek		
Data bus 16 bit olsaydı kernel'in bus cycle sayısı		
Mevcut / 16-bit oranı		

PCIe paket verimi (N × 4 = ____ byte'lık tek Memory Write TLP; Fig. 3.25):

Alan (octet)	12-byte header, ECRC yok	16-byte header, ECRC var
STP framing (baş)		
Sequence number		
Header		
Data		
ECRC		
LCRC		
STP framing (son)		
Toplam		
Verim = data / toplam		
128b/130b ile verim		

Sonucun bir çekirdekten PCIe cihazına yolu (Fig. 3.17 ve 3.21): her adımda bağlantı, protokol katmanı ve aktarım birimi.

Adım 4 — Cache simülasyonu (final paketiyle: 16 Kasım, Pazartesi)

4a. Data adres izi. Tüm load ve store'lar depoda trace.csv dosyasındadır. Biçim: sira,RW,adres_hex,boyut,c_satiri,stack_mi.

Ölçü	Değer
Kullanılan derleme (-O0 / -O1)	
Toplam data referansı	
Load / store	
Stack erişimi sayısı	
Ana diziye erişim sayısı	

4b. İlk 5 farklı adresin alanları:

Adres (hex)	Adres (binary)	Tag	Line / Set	Word

4c. Elle simülasyon (ilk 30 referans):

#	R/W	Adres (hex)	Tag	Line / Set	Word	Hit / Miss	Yerinden çıkan blok	Bu blok daha önce cache'te miydi?
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								

#	R/W	Adres (hex)	Tag	Line / Set	Word	Hit / Miss	Yerinden çıkan blok	Bu blok daha önce cache'te miydi?
12								
13								
14								
15								
16								
17								
18								
19								
20								
21								
22								
23								
24								
25								
26								
27								
28								
29								
30								

Cache'in 30. referanstan sonraki anı. Line sayınız 8'den fazlaysa satır ekleyin.

Line	Set	Valid	Tag (hex)	Dirty (write-back)	LRU sırası (associative)
0					
1					
2					
3					
4					
5					

Line	Set	Valid	Tag (hex)	Dirty (write-back)	LRU sırası (associative)
6					
7					

4d. Tam simülasyon sonuçları:

Ölçü	Elle (ilk 30)	Araç (ilk 30)	Araç (tüm iz)
Referans	30	30	
Hit			
Miss			
Hit ratio H			
Ana belleğe yazma sayısı			

Kullanılan araç ve ayarları (kendi simülatörünüz varsa dosya adı):

- ☐ Elle ve araçla yapılan simülasyon ilk 30 referansta birebir örtüşüyor.

4e. Tasarım uzayı (cache boyu sabit):

Mapping (line boyu sabit = ____ B)	Hit	Miss	H
Direct			
2-way set-associative			
4-way set-associative			
Fully associative (LRU)			

Line boyu (mapping sabit = ____)	Hit	Miss	H
8 B			
16 B			
32 B			
64 B			

Grafik dosyası ve eğilimlerin locality ile açıklaması (thrashing varsa çatışan adresler):

4f. Ortalama erişim süresi: $T_s = H \times T1 + (1 - H) \times (T1 + T2)$

Durum	H	T_s (cycle)	Speedup (cache'siz duruma göre)
Cache yok	0	T2 =	1
Kişiye özel cache			

4g. Kod değişikliği:

	Önce	Sonra
Değiştirilen C satır(lar)ı		
Y7 çıktısı (aynı olmalı)		
H		
T_s (cycle)		

Değişikliğin gerekçesi; H artmadıysa nedeni:

Adım 5 — Performans ve architecture/organization (final paketiyle: 16 Kasım, Pazartesi)

5a-1. Dinamik instruction dağılımı ve CPI (CPI_i değerleri yönergedeki model varsayımıdır):

Sınıf	CPI_i (varsayım)	I_i (-O0)	I_i (-O1)
ALU	1		
Çarpma	3		
Load	1		
Store	1		
Branch / jump	2		
Ic (toplam)			
Ortalama CPI			

5a-2. Süre ve MIPS rate (f = ____ GHz, τ = ____ ns):

Ölçü	Formül	-O0	-O1
T (bellek etkisi yok)	Ic × CPI × τ		
MIPS rate	f / (CPI × 10^6)		
m (instruction başına data referansı)	data ref. / Ic		
T, cache yok (k = T2)	Ic × [p + (m × k)] × τ		
T, cache var (k = T_s)	Ic × [p + (m × k)] × τ		

Karşılaştırma	Değer
Speedup `-O0` → `-O1` (Eq. 2.1, cache var)	
Speedup cache yok → cache var (-O0)	

MIPS rate'in iki derlemeyi karşılaştırmada neden yanıltıcı olabileceği, kendi sayılarınızla:

5a-3. Amdahl yasası (Nc = ____):

Alan	Değer
Paralelleştirilebilen kısım (C satırları)	
Bağımsızlık gerekçesi (veri bağımlılığı)	
f (yürütme süresindeki pay)	
Speedup = 1 / [(1 − f) + f / Nc]	
Nc → ∞ limiti	

5b. Architecture ve organization (en az 10 özellik):

#	Özellik	Architecture / Organization	Gerekçe (Stallings tanımı)	Değişseydi Y7 değişir miydi? Kanıt
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				

Ch. 1.2'deki dört temel fonksiyon:

Fonksiyon	Kernel'den örnek instruction (adres, assembly)	Açıklama
Data processing		
Data storage		

Fonksiyon	Kernel'den örnek instruction (adres, assembly)	Açıklama
Data movement		
Control		

Final teslim (16 Kasım Pazartesi; sözlü savunma 20 Kasım Cuma)

Teslim kontrol listesi:

- ☐ Bu tutanak, tüm adımları dolu olarak (PDF ve düzenlenebilir sürüm)
- ☐ C kaynak kodu (`main.c`) ve sonraki 4g sürümü (`main_v2.c`)
- ☐ `asm_O0.s`, `asm_O1.s` ve `objdump -d` çıktısı
- ☐ `trace.csv` ve cache simülasyon çıktıları (kendi simülatörünüz varsa kaynak kodu)
- ☐ Ekran görüntüleri: machine code doğrulaması, program çıktıları, cache durumu
- ☐ 4e grafiği
- ☐ Git deposunda M1, M2 ve final etiketleri

Kilometre taşı kaydı (öğretim üyesi doldurur):

Kilometre taşı	Son tarih	Teslim tarihi	Git etiketi	Paraf
M1 Öneri (Adım 0–1)	2 Kasım (Pzt)			
M2 Assembly ve machine code (Adım 2)	9 Kasım (Pzt)			
M3 CPU, interrupt, bus (Adım 3)	16 Kasım (Pzt, final paketiyle)			
M4 Cache ve performans (Adım 4–5)	16 Kasım (Pzt, final paketiyle)			
Final ve sözlü savunma	Teslim 16 Kasım (Pzt); savunma 20 Kasım (Cuma)			

YZ kullanım beyanı. Kullanılmadıysa ilk satıra "Kullanılmadı" yazın. Beyan edilmemiş kullanım akademik dürüstlük ihlalidir.

Araç ve sürümü	Adım	Ne için kullanıldı	Örnek prompt	Çıktıyı nasıl doğruladım
Claude (Anthropic), Claude Opus 5.5	0–1	Belgelerin özetlenmesi, görev karşılaştırması, parametre hesabının kontrolü, C kodu taslağı, Ripes kurulumu, değişken adreslerinin ve kernel'in belirlenmesi, tutanağın	"öğrenci numaram 5001230022, görev olarak G3'ü yapalım"; "bu kodu Ripes'e nasıl yapıştıracağım, adım adım söyle"	Parametreleri elle yeniden hesapladım; kodu Ripes'te (RV32IM, single-cycle) derleyip çalıştırdım ve checksum, decrypt_ok, msg[] değerlerini Memory görünümünden okudum; kodu Visual Studio'da da çalıştırdım.

Araç ve sürümü	Adım	Ne için kullanıldı	Örnek prompt	Çıktıyı nasıl doğruladım
		doldurulması		

Öğrenci beyanı. Bu tutanaktaki C kodunu, elle machine code kodlamasını, instruction cycle izleme tablosunu ve elle cache simülasyonunu kendim yaptım. Başka bir öğrencinin çalışmasını kopyalamadım ve kendi çalışmamı paylaşmadım. YZ kullanımımı eksiksiz beyan ettim. Sözlü savunmada bu çalışmayı açıklayamamamın puanımı etkileyeceğini biliyorum.

Ad Soyad	Öğrenci numarası	Tarih	İmza
Ali Paşaoğlu	5001230022		

Değerlendirme (öğretim üyesi doldurur)

Bileşen	Azami	Puan	Not
A1 Öneri ve parametre hesabı (Adım 0–1)	5		
A2 Assembly ve machine code (Adım 2)	15		
A3 CPU, interrupt, bus (Adım 3)	25		
A4 Cache (Adım 4)	25		
A5 Performans ve architecture/organization (Adım 5)	10		
Tutanak ve rapor kalitesi	5		
Sözlü savunma	15		
Geç teslim kesintisi	–		
Toplam	**100**		

Sözlü savunma kaydı:

Madde	Sorulan	Sonuç (başarılı / kısmen / başarısız)	Puan
1. Canlı encode (-O1 listesinden)			
2. Adres alanları ve hit/miss			
3. Interrupt k+1'de gelseydi			
4. Tasarım kararı savunması			

☐ 1. ve 2. maddelerin ikisi de başarısız: A2–A4 puanları 0.5 ile çarpıldı.

Öğretim üyesi	Tarih	İmza